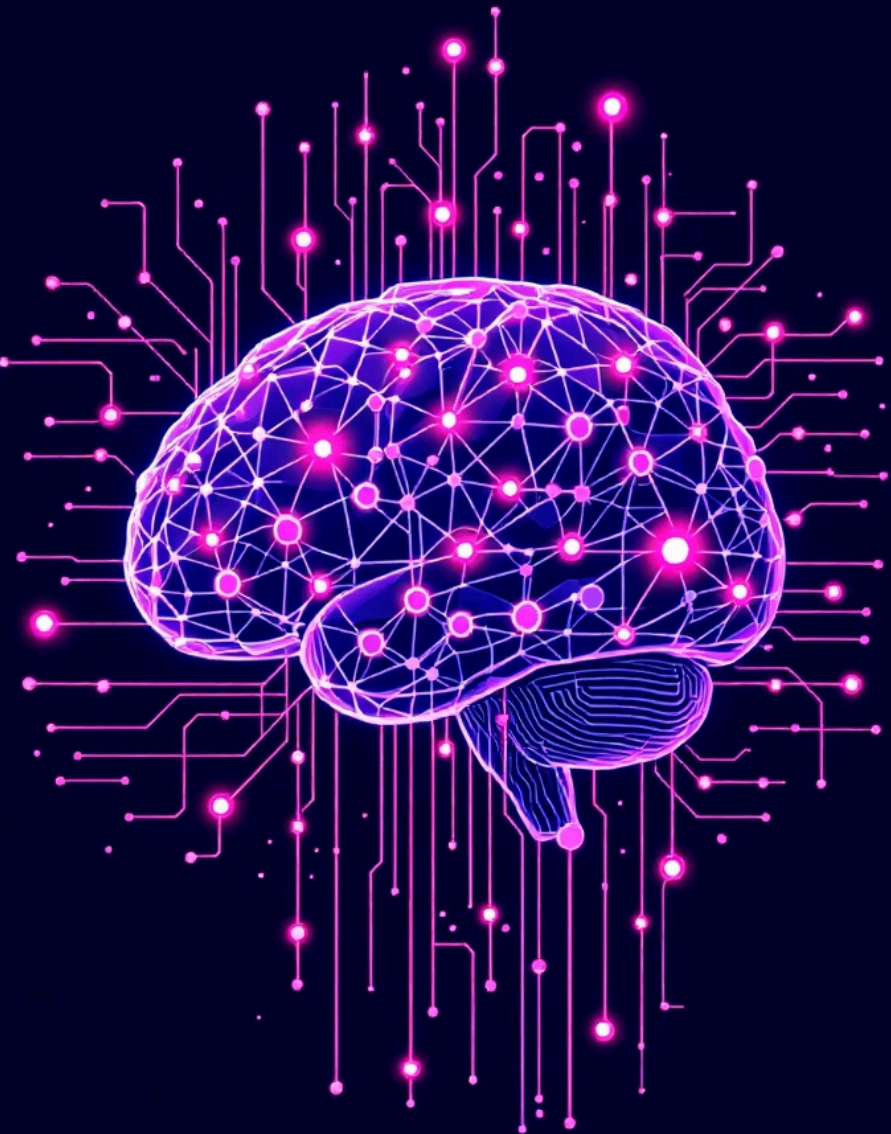




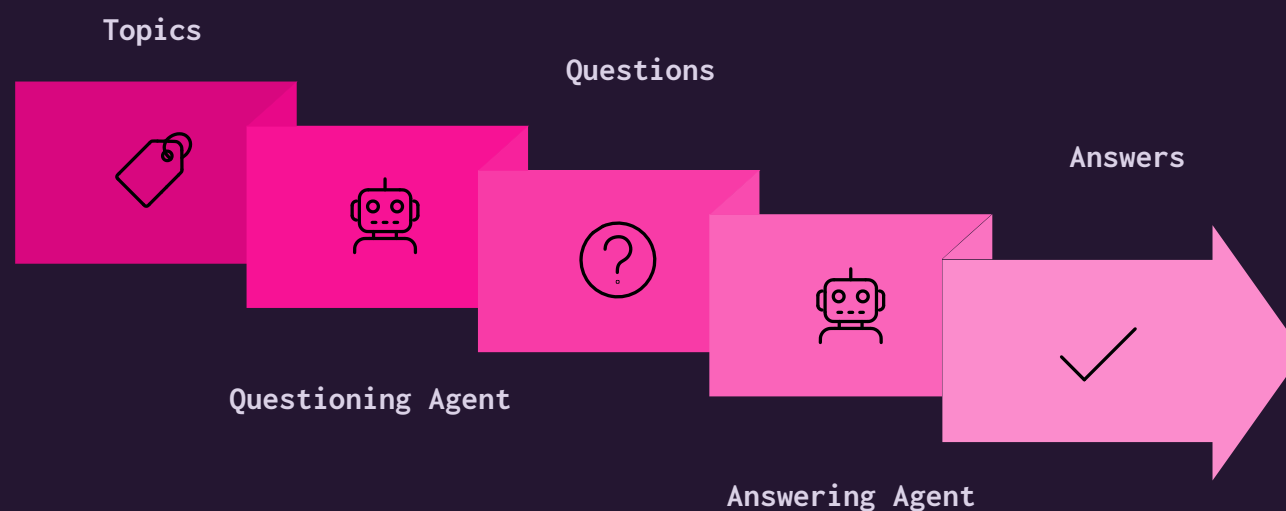
MCQ System – Core Techniques

A Compact Overview of Dual-Agent Architecture & Methods

Dual-Agent Architecture for Automated Assessments

Our MCQ system operates on a robust dual-agent architecture, designed for clarity, efficiency, and independent evolution of its core components.

- **Questioning Agent:** Responsibly generates Multiple Choice Questions (MCQs) from diverse topics, leveraging In-Context Learning (ICL) examples to maintain quality.
- **Answering Agent:** Provides accurate answers to generated MCQs, accompanied by detailed reasoning processes.
- **LLM Integration:** Both agents seamlessly wrap Large Language Model (LLM) backends (QAgent, AAgent), utilizing their advanced inference capabilities.
- **Model:** Qwen/Qwen2.5-14B-Instruct - Powers both agents with advanced language understanding and generation capabilities.
- **Modular Design:** The system's modularity ensures that each agent can be improved, scaled, or replaced independently without affecting the other, fostering continuous development.



This diagram illustrates the modular flow, where distinct agents handle question generation and answering, enabling specialized optimization.

Questioning Agent: Precision in Prompt Engineering

Expert Examiner Persona

Utilizes a system prompt like "expert examiner" to guide the LLM in crafting challenging and well-structured MCQs, ensuring high-quality content.

Strict JSON Output

Enforces a precise JSON output format, including topic, question, choices, correct answer, and explanation, for seamless integration and parsing.

Bias Mitigation via Randomization

The correct option (A/B/C/D) is randomized for each question, actively preventing positional bias in the generated assessments.

Dynamic In-Context Learning (ICL)

Relevant examples from `assets/topics_example.json` are dynamically inserted into the prompt, guiding the LLM to generate more coherent and contextually appropriate questions.

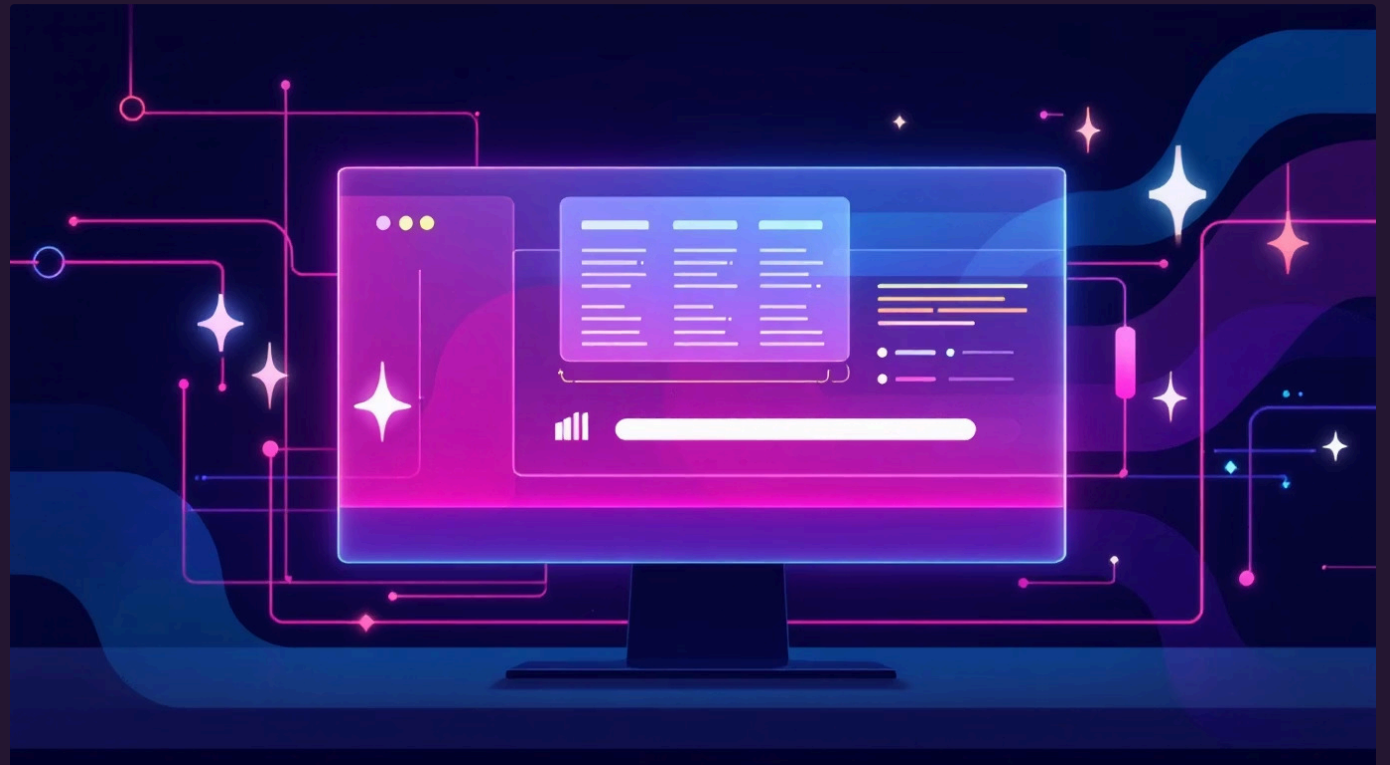


Batch Generation for Efficiency

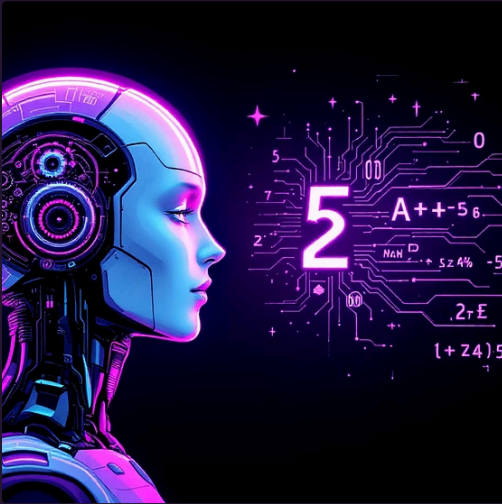
To optimize performance and manage API rate limits, the Questioning Agent employs efficient batch processing for question generation.

- **Batch Generation:** Questions are generated in batches, significantly reducing overhead per query and improving overall throughput.
- **Progress Tracking:** Integration with `tqdm` provides real-time progress bars, offering clear visibility into the generation process for large datasets.

This approach ensures that even extensive question sets can be processed quickly and reliably, providing a smooth user experience.

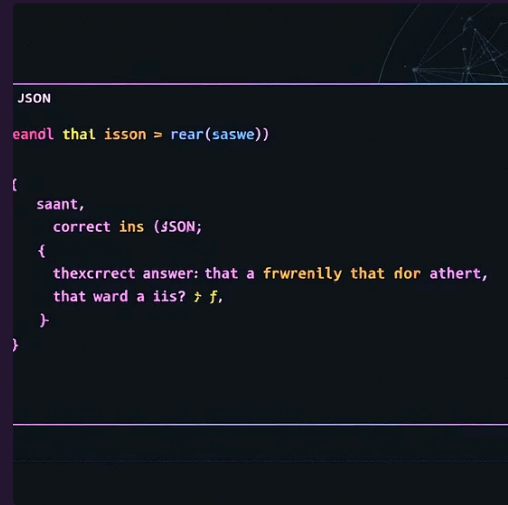


Answering Agent: Intelligent Responses & Self-Correction



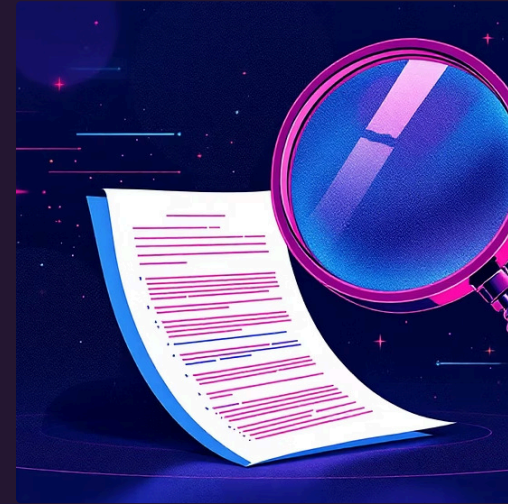
Quantitative Aptitude Expert

The agent receives a system prompt to act as an "expert in quantitative aptitude", emphasizing detailed, step-by-step reasoning for each answer.



Standardized Output

Adheres to a consistent output format: {"answer": "A", "reasoning": "..."}, facilitating easy data extraction and display.



Robust Self-Correction (Reflection)

Includes a reflection mechanism where a secondary prompt is invoked to extract the correct JSON format if the initial LLM output is malformed, enhancing reliability.



Answer Validation

Answers undergo rigorous validation, checking for correct letter format and ensuring reasoning token length is within limits (<512 tokens), maintaining conciseness and quality.

Shared Techniques Across Agents

Both the Questioning and Answering Agents benefit from a set of common techniques that ensure efficiency, reliability, and maintainability across the entire system.

1

Batch Processing

Optimizes LLM calls by processing multiple requests in parallel, effectively managing API rate limits and maximizing throughput.

2

Token Counting & Logging

Utilizes the model's tokenizer for accurate token counting, enabling filtering of overly long responses and detailed performance logging (Tokens Generated Per Second - TGPS).

3

Validation & Filtering

Implemented at multiple stages to ensure generated content adheres to required keys, correct formats, and specified token limits, preventing malformed outputs.

4

YAML Configuration

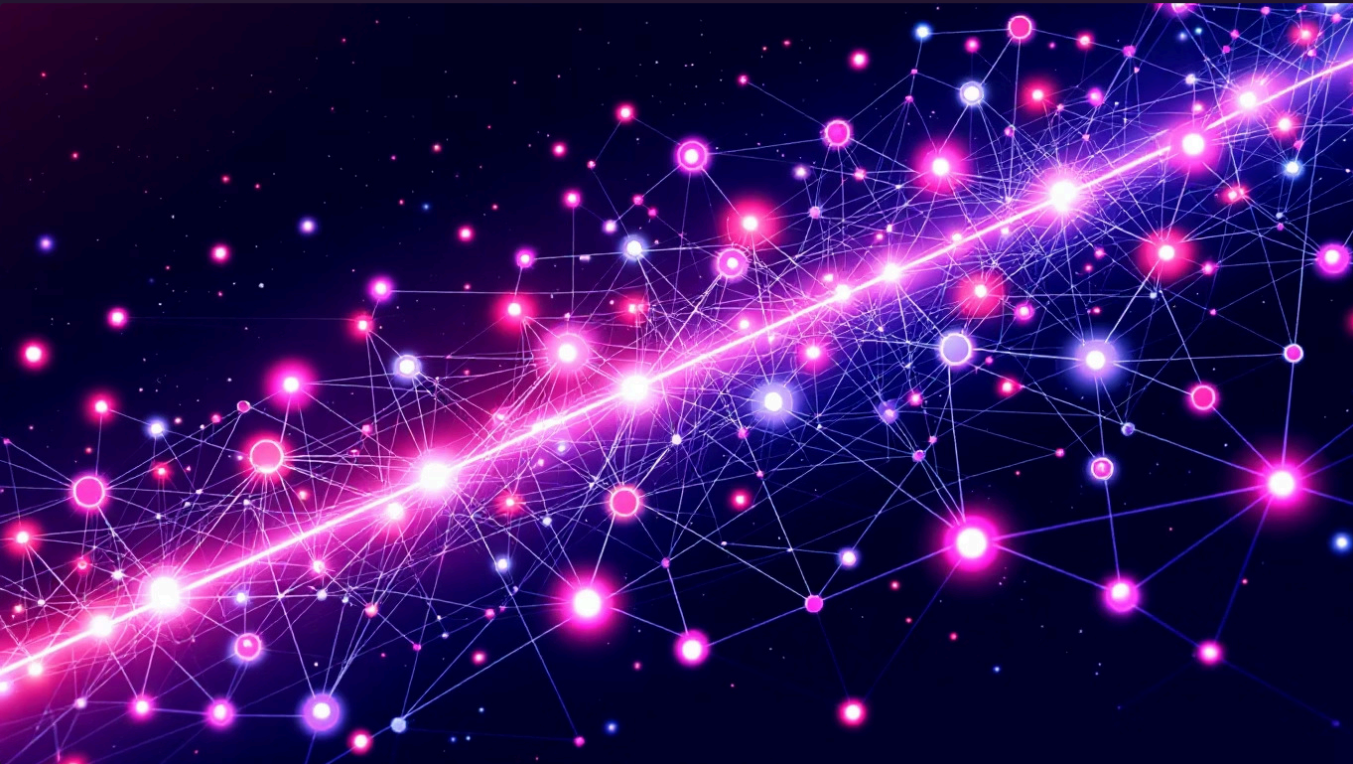
Key parameters such as temperature, `max_new_tokens`, and `tgps_show` are managed via external YAML files, simplifying configuration and enabling flexible adjustments.

5

Command-Line Interface (CLI)

Provides an intuitive CLI using `argparse` for easy interaction and integration, allowing users to control parameters like `--num_questions` or `--batch_size`.

Enhancing System Intelligence



As the system evolves, several key areas are targeted for future development to further refine its capabilities and expand its utility:

- **Adaptive Reflection:** Implementing more sophisticated self-correction mechanisms that can dynamically adjust based on the nature of errors.
- **Difficulty Tuning:** Developing methods to precisely control the difficulty level of generated MCQs, catering to different assessment needs.
- **Web Interface:** Creating a user-friendly web interface to streamline interaction, configuration, and monitoring of the MCQ generation and answering process.
- **Multilingual Support:** Expanding the system's capabilities to generate and answer MCQs in multiple languages.

Key Takeaways: A Robust Assessment Framework

Modular & Scalable Agents

Independent Questioning and Answering Agents ensure system scalability and ease of maintenance.

High-Quality Generation

Advanced prompt engineering coupled with In-Context Learning drives the creation of relevant and challenging MCQs.

Reliable & Validated Outputs

Rigorous validation and self-correction mechanisms guarantee the accuracy and proper formatting of all generated content.

Optimized Performance

Batch processing and detailed token tracking contribute to efficient operation and resource management.

Bias Mitigation

Strategic randomization of answer options minimizes positional bias, leading to fairer assessments.

Why This Approach Matters

Our dual-agent MCQ system represents a significant step forward in automated assessment technology. By emphasizing modularity, intelligent prompt engineering, and robust validation, we create a reliable, scalable, and adaptable platform for generating and evaluating knowledge.

This architecture allows for continuous improvement in specific areas without compromising the integrity of the overall system, fostering innovation in educational and training assessment.

This structured approach ensures not only high-quality output but also provides a clear pathway for future enhancements, making it a powerful tool for various applications.



Thank You

For more information or to discuss collaboration, connect with us!

Madhav Dhamija - dhamijamadhav3@gmail.com

Sujay Kumar - sujaykumar12@gmail.com

Sumeet Dutta - sumeetdutta040@gmail.com

Sahil Sumrani - officialsahilarora05@gmail.com

